

Supplementary Material

When the Same Question Gets Different Answers:
Quantifying LLM Non-Determinism in Evidence Synthesis

Lucas Rover

Yara de Souza Tadano

Contents

1 PubMed Search Strategy	2
1.1 Broad Query (573 records)	2
1.2 Design-Filtered Query (222 records)	2
1.3 Exclusion Candidates (475 records)	2
1.4 Corpus Assembly	2
2 Prompt Templates	3
2.1 Stage A: Abstract Screening Prompt	3
2.2 Stage B: Data Extraction Prompt	4
3 JSON Schemas for Output Validation	4
3.1 Screening Output Schema	5
3.2 Extraction Output Schema	5
4 Gold Standard Labeling Protocol	5
4.1 Screening Labels	5
4.2 Extraction Labels	5
5 Example Outputs: Exact Match vs. Near-Miss	5
5.1 Example 1: Exact Match (Local Model – Gemma 2 9B)	6
5.2 Example 2: Near-Miss (Cloud API – Claude Sonnet 4.5)	6
6 Model Configurations	7
7 Provenance Protocol Details	7
7.1 Call-Level Hashing	7
7.2 Output Hashing	8
7.3 Run Cards	8
7.4 Verification Protocol	8
8 Corpus Statistics	8
9 Experimental Timeline	8
10 Software and Dependencies	8
11 Complete Bootstrap Confidence Intervals	9

1 PubMed Search Strategy

All abstracts were retrieved from PubMed via the NCBI E-utilities API (`esearch` + `efetch`) on February 11, 2026. Two complementary queries were used:

1.1 Broad Query (573 records)

Listing 1: PubMed broad search query

```
("Particulate Matter"[MeSH] OR "PM2.5" OR "fine particulate matter"
OR "fine particles" OR "ambient particulate matter")
AND ("Hospitalization"[MeSH] OR "hospital admission"
OR "hospital admissions" OR "emergency department"
OR "ED visit" OR "ED visits")
AND ("Respiratory Tract Diseases"[MeSH] OR "respiratory"
OR "asthma" OR "COPD" OR "pneumonia" OR "bronchitis"
OR "bronchiolitis")
AND ("time series" OR "time-series" OR "case-crossover"
OR "distributed lag" OR "Poisson regression" OR "GAM"
OR "generalized additive model")
```

1.2 Design-Filtered Query (222 records)

Listing 2: PubMed design-filtered query

```
("PM2.5" OR "fine particulate matter"
OR "particulate matter 2.5")
AND ("respiratory" OR "asthma" OR "COPD" OR "pneumonia")
AND ("hospitalization" OR "hospital admission"
OR "emergency department")
AND ("time series" OR "time-series" OR "case-crossover")
```

1.3 Exclusion Candidates (475 records)

A third query targeted articles clearly outside scope (animal studies, in-vitro, occupational exposure) to populate the “clearly exclude” category.

1.4 Corpus Assembly

From the merged candidate pool (1,021 unique records after deduplication):

- 100 abstracts meeting all six inclusion criteria were labeled “clearly include”
- 100 abstracts with unambiguous exclusion reasons were labeled “clearly exclude”
- 300 abstracts with borderline characteristics (e.g., mixed pollutants, ambiguous outcomes) were labeled “ambiguous”

The 200 “clear” abstracts were independently labeled by two reviewers with discordance resolution. The 300 “ambiguous” abstracts were classified using study-design indicators and keyword matching (see Labeling Guide, Section 4).

2 Prompt Templates

The exact prompts used for both pipeline stages are reproduced below. These prompts were held constant across all 6 models and all 120 experimental runs.

2.1 Stage A: Abstract Screening Prompt

Listing 3: Screening prompt template (configs/prompts/screening.txt)

```
You are an expert epidemiologist conducting a systematic review on
the association between PM2.5 exposure and respiratory
hospitalizations.

Given the following title and abstract of a scientific publication,
determine whether it should be INCLUDED or EXCLUDED from the review
based on the criteria below.

## Inclusion Criteria
1. Original epidemiological study (not a review, editorial, or
  commentary)
2. Exposure: PM2.5 or fine particulate matter (<=2.5 um)
3. Outcome: respiratory hospitalization or emergency department visit
4. Study design: time-series or case-crossover
5. Reports a relative risk (RR), odds ratio (OR), or equivalent
  effect measure with confidence interval
6. Published in English

## Exclusion Criteria
- Reviews, meta-analyses, editorials, or commentaries
- Exposure other than PM2.5 (unless PM2.5 is one of multiple
  exposures analyzed)
- Outcome other than respiratory hospitalization/ED visit
- No quantitative effect estimate reported
- Animal or in-vitro studies

## Input
Title: {title}
Abstract: {abstract}

## Instructions
Respond with a JSON object only. Do not include any text outside
the JSON.

{
  "decision": "include" | "exclude" | "uncertain",
  "confidence": 0.0-1.0,
  "rationale": "brief explanation of decision",
  "exposure": "PM2.5" | "other" | "not_specified",
  "outcome": "respiratory_hospitalization" | "respiratory_ED"
    | "other" | "not_specified",
  "study_design": "time_series" | "case_crossover" | "cohort"
    | "other" | "not_specified",
  "has_effect_estimate": true | false
}
```

2.2 Stage B: Data Extraction Prompt

Listing 4: Extraction prompt template (configs/prompts/extraction.txt)

```
You are an expert epidemiologist extracting quantitative data from a
scientific publication on PM2.5 and respiratory hospitalizations for
a systematic review and meta-analysis.

Given the following title and abstract, extract the structured data
below. If a field is not reported in the abstract, use null.

## Input
Title: {title}
Abstract: {abstract}

## Instructions
Extract ALL reported effect estimates. If multiple estimates are
reported (e.g., different lags, subgroups), extract each as a
separate entry in the "estimates" array.

Respond with a JSON object only. Do not include any text outside
the JSON.

{
  "study_id": "first_author_year",
  "study_location": "city, country",
  "study_period": "YYYY-YYYY",
  "study_design": "time_series" | "case_crossover" | "other",
  "population": "general" | "elderly" | "children" | "other",
  "sample_size": null | integer,
  "estimates": [
    {
      "effect_measure": "RR" | "OR" | "HR" | "IRR" | "other",
      "effect_estimate": float,
      "ci_lower": float,
      "ci_upper": float,
      "ci_level": 95,
      "exposure_increment": "per 10 ug/m3" | "per IQR" | "other",
      "lag": "lag0" | "lag1" | "lag0-1" | "other",
      "outcome_specific": "all_respiratory" | "pneumonia"
                          | "COPD" | "asthma" | "other",
      "covariates": ["temperature", "humidity", "trend",
                    "seasonality", "..."]
    }
  ]
}
```

3 JSON Schemas for Output Validation

All LLM outputs were validated against JSON Schema (Draft-07) definitions. Outputs failing validation were flagged but retained to ensure complete provenance.

3.1 Screening Output Schema

Seven required fields, no additional properties permitted: `decision` (enum: include/exclude/uncertain), `confidence` (0.0–1.0), `rationale` (string, min 10 chars), `exposure`, `outcome`, `study_design` (categorical enums), and `has_effect_estimate` (boolean).

3.2 Extraction Output Schema

Required fields: `study_id` (string), `estimates` (array). Optional metadata: `study_location`, `study_period`, `study_design`, `population`, `sample_size`. Each estimate requires: `effect_measure`, `effect_estimate`, `ci_lower`, `ci_upper` (all with range constraints 0.1–10.0).

Full schemas are available in the repository at `configs/schemas/`.

4 Gold Standard Labeling Protocol

4.1 Screening Labels

The 200 “clear” abstracts (100 include, 100 exclude) were independently labeled by two reviewers following a structured protocol. Discordance was resolved by a third reviewer. Inter-rater agreement targets: Cohen’s $\kappa \geq 0.80$, discordance rate $< 15\%$.

The 300 “ambiguous” abstracts were classified using keyword matching and study-design indicators. These labels serve as reference values for accuracy computation but do not affect the primary metric (EMR), which measures self-consistency regardless of ground truth.

4.2 Extraction Labels

For the 100 “clearly include” articles, extraction templates were created following standardized rules:

- Prefer the primary/main estimate if multiple are reported
- Prefer all-respiratory over specific conditions (asthma, COPD)
- Prefer single-pollutant model over multi-pollutant
- Prefer lag 0–1 if multiple lags reported without a “best” designation
- Record per 10 $\mu\text{g}/\text{m}^3$ – convert if different increment used
- If abstract reports % change, convert: $\text{RR} = 1 + (\% \text{change}/100)$

Discordance threshold: > 0.005 for effect estimates, > 0.01 for CI bounds.

5 Example Outputs: Exact Match vs. Near-Miss

To illustrate the nature of extraction non-determinism, we present two representative cases.

5.1 Example 1: Exact Match (Local Model – Gemma 2 9B)

All 10 runs produced identical output (EMR = 1.000):

Listing 5: Gemma 2 9B extraction output (identical across all 10 runs)

```
{
  "study_id": "chen_2017",
  "study_location": "Shanghai, China",
  "study_period": "2013-2015",
  "study_design": "time_series",
  "population": "general",
  "sample_size": null,
  "estimates": [
    {
      "effect_measure": "RR",
      "effect_estimate": 1.023,
      "ci_lower": 1.005,
      "ci_upper": 1.042,
      "ci_level": 95,
      "exposure_increment": "per 10 ug/m3",
      "lag": "lag0-1",
      "outcome_specific": "all_respiratory",
      "covariates": ["temperature", "humidity", "day_of_week",
                    "long_term_trend", "seasonality"]
    }
  ]
}
```

5.2 Example 2: Near-Miss (Cloud API – Claude Sonnet 4.5)

Runs 1–8 and 10 produced one output; Run 9 differed in the `estimates` array:

Listing 6: Claude Sonnet 4.5 – Run 9 extracted 2 estimates instead of 1

```
{
  "study_id": "chen_2017",
  "study_location": "Shanghai, China",
  "study_period": "2013-2015",
  "study_design": "time_series",
  "population": "general",
  "sample_size": null,
  "estimates": [
    {
      "effect_measure": "RR",
      "effect_estimate": 1.023,
      "ci_lower": 1.005,
      "ci_upper": 1.042,
      "ci_level": 95,
      "exposure_increment": "per 10 ug/m3",
      "lag": "lag0-1",
      "outcome_specific": "all_respiratory",
      "covariates": ["temperature", "humidity", "day_of_week",
                    "long_term_trend", "seasonality"]
    },
  ],
}
```

```

{
  "effect_measure": "RR",
  "effect_estimate": 1.031,
  "ci_lower": 1.008,
  "ci_upper": 1.054,
  "ci_level": 95,
  "exposure_increment": "per 10 ug/m3",
  "lag": "lag2",
  "outcome_specific": "pneumonia",
  "covariates": ["temperature", "humidity", "day_of_week"]
}
]
}

```

Key observation: The metadata fields (`study_id`, `study_location`, `study_design`) are identical across all 10 runs (consistent with BERTScore $F_1 = 0.997$). The variation lies in the `estimates` array—Run 9 extracted an additional sub-group estimate (pneumonia at lag 2) that the other 9 runs did not. This single additional estimate causes the hash-based EMR to register a non-match, while the BERTScore over metadata text remains near-perfect. This exemplifies the “semantic paradox” reported in Section 4.5 of the main paper.

6 Model Configurations

Table 1: Complete model configurations for all six LLMs.

Model	Provider	Temp.	Seed	Max Tokens	Timeout	Version
LLaMA 3 8B	Ollama	0.0	42	4096	600s	llama3:8b
Mistral 7B	Ollama	0.0	42	4096	600s	mistral:7b
Gemma 2 9B	Ollama	0.0	42	4096	600s	gemma2:9b
Claude Sonnet 4.5	Anthropic	0.0	—	4096	90s	claude-sonnet-4-5-20250929
Gemini 2.5 Pro	Google	0.0	42	8192 [†]	90s	gemini-2.5-pro
GPT-4.1	OpenAI	0.0	42	4096	90s	gpt-4.1

[†]Gemini requires higher `maxOutputTokens` because thinking tokens consume the output budget.

7 Provenance Protocol Details

7.1 Call-Level Hashing

Each LLM call produces a **call hash** computed as:

Listing 7: Call hash computation (`src/provenance/hasher.py`)

```

import hashlib, json

def compute_call_hash(prompt, input_text, model_id, temperature,
                      seed=None):
    fields = {
        "prompt": prompt,
        "input_text": input_text,
        "model_id": model_id,

```

```

        "temperature": temperature,
        "seed": seed,
    }
    canonical = json.dumps(fields, sort_keys=True,
                           ensure_ascii=True)
    return hashlib.sha256(canonical.encode()).hexdigest()

```

The use of `sort_keys=True` and `ensure_ascii=True` ensures deterministic serialization across platforms and Python versions.

7.2 Output Hashing

The raw LLM response text is hashed directly:

```

def compute_output_hash(output_text):
    return hashlib.sha256(output_text.encode()).hexdigest()

```

7.3 Run Cards

Each run produces a structured JSON run card containing:

- Run metadata: model ID, provider, stage, run number
- Configuration: temperature, seed, max tokens
- Execution: timestamps, call counts (total/success/fail), duration
- Provenance: aggregate output hash (SHA-256 of sorted individual hashes joined by pipe delimiter)

7.4 Verification Protocol

If two runs of the same model produce different aggregate hashes, the per-call output hashes are compared to identify exactly which abstracts changed. This enables targeted analysis of non-determinism at the individual-abstract level.

8 Corpus Statistics

9 Experimental Timeline

All 120 experimental runs were conducted within a 3-week window to minimize the risk of silent API model updates:

10 Software and Dependencies

- **Python:** 3.14.3
- **OS:** macOS Darwin 24.6.0 (Apple M4, 24 GB RAM)
- **Ollama:** v0.15.5 (local model serving)

Table 2: Corpus descriptive statistics.

Statistic	Value
Total abstracts	500
Unique journals	148
Publication range	1994–2026
Mean abstract length (characters)	1,873
Median abstract length (characters)	1,812
Min abstract length (characters)	423
Max abstract length (characters)	4,291
Clear include	100 (20%)
Clear exclude	100 (20%)
Ambiguous	300 (60%)

Table 3: Experiment execution timeline.

Model	Screening	Extraction	Duration
LLaMA 3 8B	Feb 11–12	Feb 12–13	~91h
Mistral 7B	Feb 13–15	Feb 15–16	~77h
Gemma 2 9B	Feb 16–17	Feb 17	~46h
Claude Sonnet 4.5	Feb 18	Feb 18–19	~12h
Gemini 2.5 Pro	Feb 20	Feb 20–21	~8h
GPT-4.1	Feb 22	Feb 22–Mar 2	~6h

- **Key dependencies:** `numpy` ≥ 1.26 , `pandas` ≥ 2.1 , `scipy` ≥ 1.12 , `jsonschema` ≥ 4.20 , `bert-score` $\geq 0.3.13$, `transformers` ≥ 4.36
- **No external SDKs:** All API calls use Python’s `urllib.request` to ensure uniform request handling
- **Test suite:** 45 tests (`pytest` ≥ 8.0), 100% pass rate
- **License:** MIT
- **Repository:** <https://github.com/Roverlucas/llm-evidence-synthesis-reproducibility>

11 Complete Bootstrap Confidence Intervals

All confidence intervals were computed using the percentile bootstrap method with 10,000 resamples. For models with $\text{EMR} = 1.000$, the bootstrap trivially returns $[1.000, 1.000]$; the rule-of-three upper bound on the non-match rate is $3/n$ (0.6% for $n = 500$ screening, 3.0% for $n = 100$ extraction).

Table 4: Complete EMR with 95% bootstrap confidence intervals for all model-stage combinations.

Model	Stage	EMR	95% CI	n
LLaMA 3 8B	Screening	1.000	[1.000, 1.000]*	480
LLaMA 3 8B	Extraction	1.000	[1.000, 1.000]*	96
Mistral 7B	Screening	1.000	[1.000, 1.000]*	500
Mistral 7B	Extraction	1.000	[1.000, 1.000]*	100
Gemma 2 9B	Screening	1.000	[1.000, 1.000]*	500
Gemma 2 9B	Extraction	1.000	[1.000, 1.000]*	100
Claude Sonnet 4.5	Screening	0.974	[0.960, 0.986]	500
Claude Sonnet 4.5	Extraction	0.050	[0.010, 0.090]	100
Gemini 2.5 Pro	Screening	0.936	[0.914, 0.956]	500
Gemini 2.5 Pro	Extraction	0.200	[0.120, 0.280]	100
GPT-4.1	Screening	0.970	[0.954, 0.984]	500
GPT-4.1	Extraction	0.150	[0.080, 0.220]	100

*Rule-of-three upper bound on non-match rate: $3/n$ (screening: 0.6%, extraction: 3.0%).